

# DeepSeek-V4 技术全景浅解

- 迈向百万级 Token 上下文智能与极致效率革命
- 极其细致的架构、优化器与训练演进报告
- 核心技术改进 (Before vs After) 全面对比
- 报告生成时间: 2026.04 ▪ AIGCode
- 参考: [https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek\\_V4.pdf](https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf)

## 02 | 多个微创x强基层=改造模型范式

- 模型双擎矩阵:
  - - DeepSeek-V4-Pro: 1.6T 总参数 (49B 激活参数)
  - - DeepSeek-V4-Flash: 284B 总参数 (13B 激活参数)
- 上下文能力: 原生支持 1,000,000 (1M) Token 超长上下文
- 训练规模: 基于 32 万亿 (32T) 高质量 Token 的预训练
- 三大核心突破: 混合注意力、流形约束超连接、Muon 优化器

# 03 | 关键价值与核心突破总览

- **【核心突破】**
- 推理开销断崖式下降：百万 Token 场景名单 Token 推理 FLOPs 仅需 27%
- KV Cache 极致压缩：内存占用降至前代的 10%
- Pro-Max 推理模式：解锁上限最强的逻辑推理与思维链 (CoT) 能力
- **【关键价值】**
- 打破内存墙：单机处理 1M 上下文成为商用可能
- 打破收敛瓶颈：大幅降低极深万亿网络训练成本

# 04 | V4 : Pro 与 Flash

- **【DeepSeek-V4-Pro】**
- 参数量: 1.6 Trillion (1.6万亿), 激活 49B
- 定位: 极致性能, 复杂推理与泛化
- 场景: Pro-Max 深度思考、代码生成、长文分析
- **【DeepSeek-V4-Flash】**
- 参数量: 284 Billion (2840亿), 激活 13B
- 定位: 极速推理, 高并发与低延迟
- 场景: 实时对话、快速提取、高频 API

# 05 | 三大核心技术代差

- 1. 注意力机制
  - - Before: 标准 MHA / MLA, 长文本 KV 爆炸
  - - After: Hybrid Attention (CSA+HCA), KV 压缩至 10%
- 2. 残差连接
  - - Before: 标准 Pre-Norm, 深层网络流形崩塌
  - - After: mHC 流形约束超连接, 解决信号衰减
- 3. 训练优化器
  - - Before: AdamW, 32T 数据下收敛遇颈
  - - After: Muon 优化器, 正交约束加速二阶收敛

## 06 | 百万上下文的“内存墙”挑战

- 传统 Attention 复杂度：计算量  $O(N^2)$ ，KV Cache 内存占用  $O(N)$ 。
- 百万 Token 的灾难：1M 长度下，即使是 MLA，KV Cache 也足以撑爆 8xH100。
- 痛点：推理主要处于 Memory-Bound (访存受限) 状态，算力被闲置。
- V4 的解法：彻底改变注意力物理结构，实现“选择性遗忘”与“深度压缩”。

# 07 | Before: Attention 痛点

- MHA (Multi-Head Attention): 每个 Head 保存独立的 K 和 V。1M 长度下内存完全不可接受。
- MQA / GQA: 共享 K 和 V。缓解了部分压力，但精度受损，1M 长度仍显庞大。
- MLA (V3 方案): 投影到低维潜空间。已是巨大进步，但在 1.6T 参数量下，100% 潜变量缓存依然是瓶颈。

# 08 | After: V4 混合注意力架构

- 核心特点：将传统的同质化 Attention 拆分为两种异构协作机制。
- 机制 A: CSA - 压缩稀疏注意力 ▪ 捕获近距高频特征。
- 机制 B: HCA - 深度压缩注意力 ▪ 捕获远距宏观背景。
- 协作逻辑：模型同时并行运行 CSA 与 HCA。

## DeepSeek-V4 混合注意力架构 (Hybrid Attention)

将同质化 Attention 拆分为两种异构机制，并行处理近距与远距特征





# 09 | CSA: 压缩稀疏注意力

- 稀疏性 (Sparsity): 当前 Token 只与极少数历史 Token 强关联。CSA 仅激活/加载少量 KV 块。
- 压缩性 (Compression): 结合 MLA 思想, 缓存低秩表达而非完整隐藏层。
- 【Before vs After】
  - Before: 稠密计算
    - 加载全部历史
  - After: 仅加载稀疏命中
    - 压缩 Block
    - 计算量锐减



| 维度    | Before (V3.2 / MLA) | After (V4 / HCA+CSA) |
|-------|---------------------|----------------------|
| 注意力机制 | 多头潜变量 (Fixed)       | 压缩策略注意力 (Dynamic)    |
| KV 存储 | 线性扩展 ( $O(N)$ )     | 深度压缩 (90% Reduction) |



# 10 | HCA: 深度压缩注意力

- 超高压缩率：针对不需精确到词的“宏观背景知识” ( $m' \gg m$ )。
- 池化聚合：将极长历史窗口内的 Token 聚合，形成低维“摘要 Token”。
- **【Before vs After】**
  - - Before: 全局注意力需遍历所有远距离 Token。
  - - After: 将数十万远距离 Token 压缩为几百个高阶摘要 Token。
- 协同路由策略：
  1. 近端窗口 (Local Window): 精确无损注意力，保留极高分辨率语义。
  2. 中远端稀疏 (CSA Region): 精准召回所需历史块，摒弃无效信息。
  3. 极远端宏观 (HCA Region): 利用深度压缩态获取全局上下文。
- 结果：三者输出加权求和，既见树木，又见森林。
- 对比基准：DeepSeek-V3.2 在 1M Token 时的 KV Cache 为 100%。
- V4 表现：由于 HCA 极高压缩比和 CSA 稀疏卸载，驻留显存的 KV Cache 仅为 V3.2 的 10%。
- 物理意义：8卡 H100 才能装下的 1M 上下文，现在单卡即可承载！
- 影响：极大降低了长文本商业化的硬件门槛。

# 11 | 突破：推理 FLOPs 降至 27%

- 计算量重塑：Attention 的计算复杂度被彻底打破。
- Before (V3.2): 新 Token 需与 1M 个历史 Token 进行点积。
- After (V4): 新 Token 仅与 CSA 稀疏召回的少数 Token, 及 HCA 的宏观 Token 点积。
- 结果：单 Token 推理的计算量 (FLOPs) 锐减了 73% (即为 V3.2 的 27%)。
- 长文本服务优化：避免共享前缀的重复 Prefill 计算。磁盘缓存
- CSA/HCA 存储：直接将压缩后的 KV 落盘，命中时直接复用。
- SWA 缓存策略：
  - - Full SWA Caching: 零冗余但写入密集。
  - - Periodic Checkpointing: 定期快照，平衡存储与计算。
  - - Zero SWA Caching: 不存 SWA，依赖压缩 KV 重算近端。

# 12 | 工程： 底层算子与异构存储

- Kernel 优化: 重写 Triton/CUDA 算子, 让 CSA/HCA 跑满硬件带宽。
- 混合精度存储: RoPE 维度用 BF16, 其余维度用 FP8, KV Cache 再次减半。
- FP4 计算加速: CSA Indexer 的 QK 路径采用 FP4 量化, 极致加速。
- 异步预取: 预测下一组稀疏 Block, 利用 NVLink 异步加载掩盖延迟。
- 仍用RoPE是落后的, 借我们ICML2026的TPE对位置编码基础研究

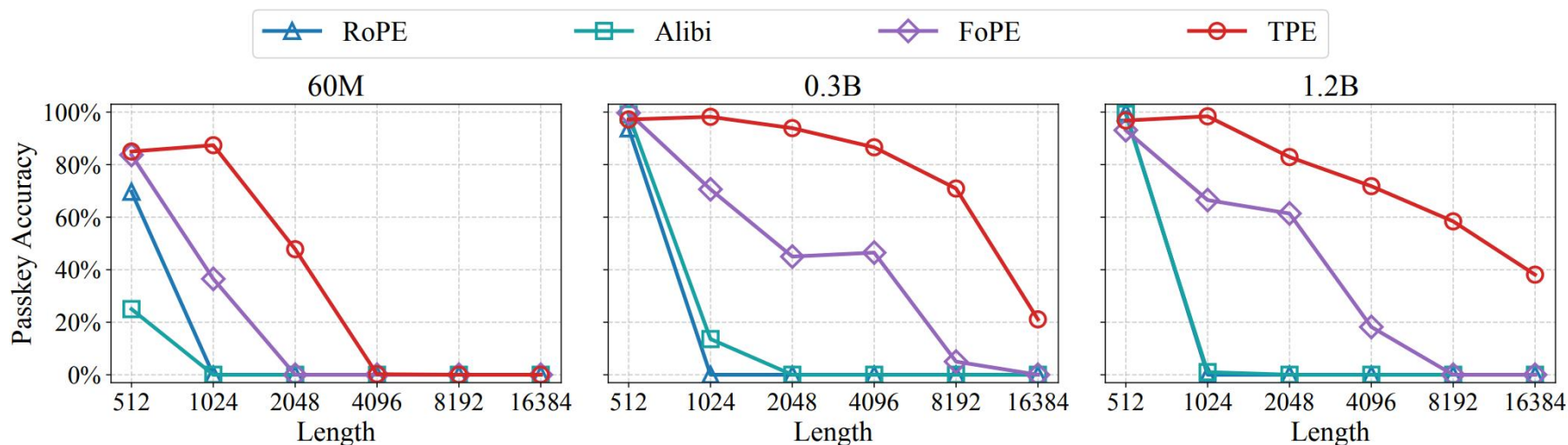


Figure 3. Passkey retrieval accuracy (%) across positional encoding methods for 60M, 0.3B, and 1.2B parameter models.

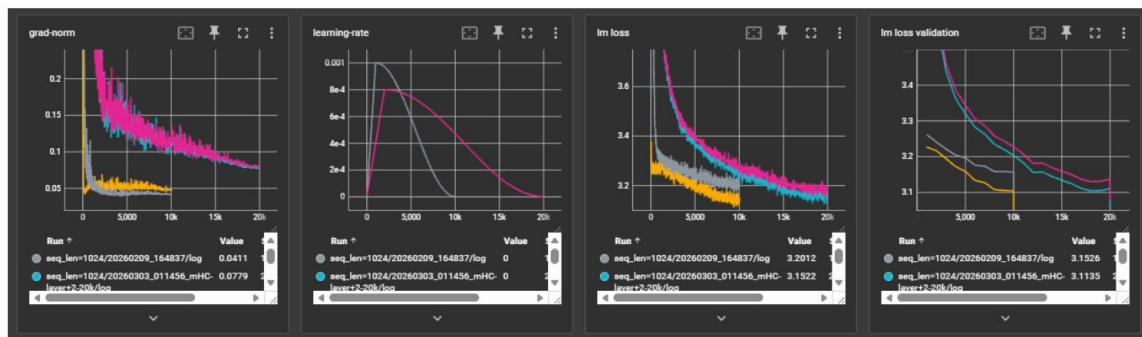
# 13 | WHY 1.6T要“连接革命”？

- 万亿模型的诅咒：网络深度和宽度达到 V4-Pro 级别时，信息传递极其脆弱。
- 梯度稀释：反向传播时，梯度在极深网络中呈指数级衰减。
- 流形崩塌 (Manifold Collapse)：隐藏层特征在多层残差映射后，退化到低维子空间，丧失表达能力。
- 诉求：需要一种全新的跨层连接拓扑。
- Post-Norm:
  - - 公式:  $x_{l+1} = \text{LayerNorm}(x_l + F(x_l))$
  - - 缺陷: 深层梯度爆炸，在百亿参数后几乎被抛弃。
- Pre-Norm / RMSNorm:
  - - 公式:  $x_{l+1} = x_l + F(\text{RMSNorm}(x_l))$
  - - 缺陷: 主流方案。但在 100+ 层的 MoE 中，主干信号被稀释，引发崩塌。

# 14 | After: mHC流形约束连接

- mHC: Manifold-Constrained Hyper-Connections.
- 超连接 (Hyper-Connections): 打破逐层相加模式, 引入跨度更大的跳跃/动态权重连接, 强化原始输入信号。
- 流形约束 (Manifold-Constrained): 强制每一层的隐藏状态保持在特定的数学流形 (Manifold) 上。
- 目标: 防止空间坍缩与表征同质化。
- 我们AIGCode
- L2两阶段预训练实验
- 证明这跨层链接有效性

增加正常mHC训练的模型参数量 (通过增加1-2个transformer layer), 从而实现同等训练Float



- 增加1个transformer layer(粉)的lm loss为3.180, 增加2个transformer layer(蓝)的lm loss为3.152

总体结论

- 同等训练float成本下, 对比train step scaling和parameter scaling, 现v2呈现出更好的收敛效果
- 在此模型规模下, 相较于w/o init, Ours stage2 w/ init可以更加快(3.6k->600)的恢复到stage 1水平

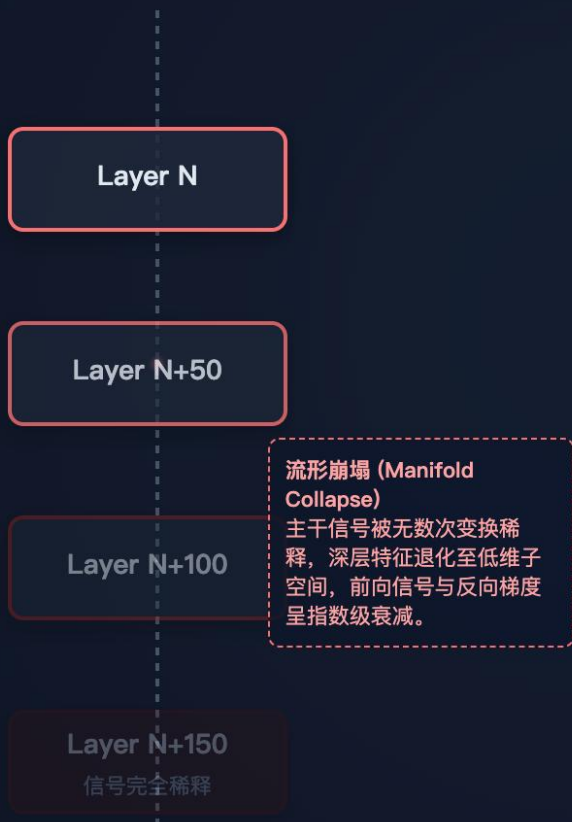


# 15 | mHC 的数学原理示意

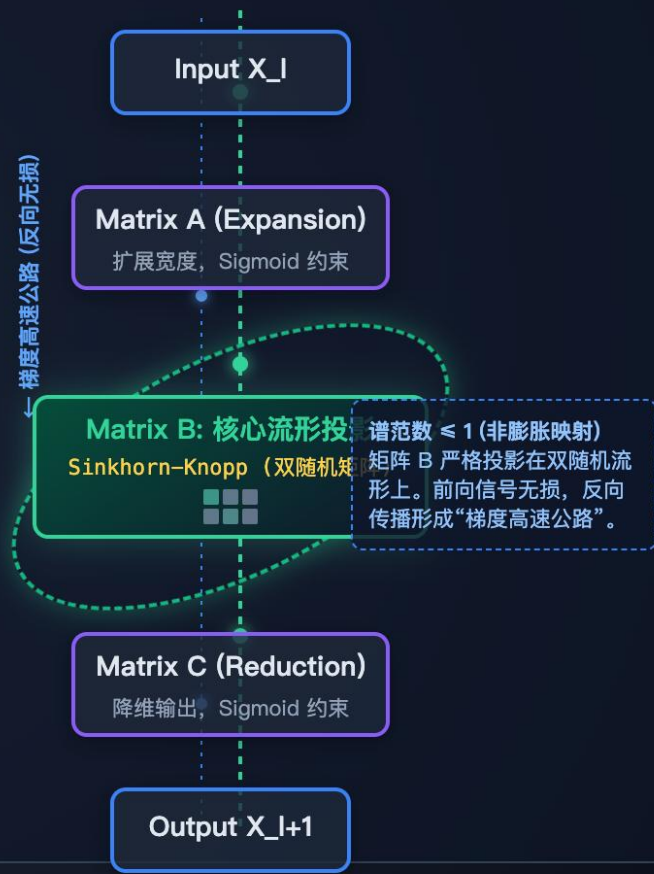
## DeepSeek-V4: 流形约束超连接 (mHC) 原理

打破万亿参数“深层梯度诅咒”，确保特征映射严格处于双随机流形之上

### Before: 传统残差 (Pre-Norm) 堆叠



### After: mHC (流形约束超连接)



# 16 | 动态参数化与 Sinkhorn-Knopp 算法

- 残差宽度扩展：将残差流宽度扩展  $n_{hc}$  倍 (如4倍)。
- 线性映射分解：引入输入映射(A)、残差变换(B)与输出映射(C)。
- 流形约束：将残差映射矩阵 B 约束在双随机矩阵 (Doubly Stochastic Matrices) 流形上。
- 效果：保证矩阵谱范数受限于 1，实现非膨胀 (Non-expansive) 映射。
- 动态生成：A, B, C 的参数分为动态（输入相关）和静态（输入无关）两部分。
- 约束应用：A 和 C 通过 Sigmoid 函数保证非负性与有界性。
- Sinkhorn-Knopp 迭代：对矩阵 B 应用指数函数并进行行列交替归一化 (迭代 20 次)，将其强投影至双随机矩阵流形。
- 稳定保障：数学上确保了极深堆叠下的绝对稳定。



# 17 | 解决信号衰减与梯度流改善

- 梯度的高速公路：mHC 就像为反向传播修建的高速公路。
- **Before:** 梯度回传 50 层后，范数衰减 90%。
- **After (mHC):** 跨层约束保证了深层损失无损传递到浅层 Embedding 层。
- 结论：初期层依然能获得充足的更新信号，加速早期收敛。
- 稳定 49B 激活参数：不同 Token 激活不同专家，易导致方差不齐引发 Loss Spike。
- 方差对齐：mHC 强制所有专家的输出流形一致。
- 根除专家坍塌：彻底解决了极大规模 MoE 训练中的激活失衡问题。
- 关键价值：使得 1.6T 模型能够平稳完成 32T Token 的漫长训练。
- 问题：mHC 增加了激活内存占用与通信量。
- 解法 1 - Fused Kernels：为 mHC 设计高度融合的算子。
- 解法 2 - 激活重计算 (Activation Checkpointing)：基于 Tensor 级别的细粒度重计算，释放缓存空间。
- 解法 3 - DualPipe 调度：隐藏额外的通信开销。
- 结果：mHC 仅带来 6.7% 的额外训练耗时，性价比极高。

# 18 | mHC 与并行策略的协同

- 在 1.6T 模型的切分中，mHC 需与 TP、PP 协同。
- 由于扩展了残差宽度，激活值通信量增加。
- DeepSeek-V4 通过优化的 All-to-All 策略，将 mHC 的开销完美隐藏在 MoE 专家的分发网络中。
- 使得理论上的复杂数学公式，在物理集群上毫无性能阻碍地运行。
- 核心痛点：万亿级深层网络的梯度消失与流形坍缩。
- 革命性解法：Manifold-Constrained Hyper-Connections。
- 突破：保证矩阵谱范数  $\leq 1$ ，实现无限深度的非膨胀传递。
- 价值：保障 1.6T 模型在 32T 数据下不发散、不崩溃。

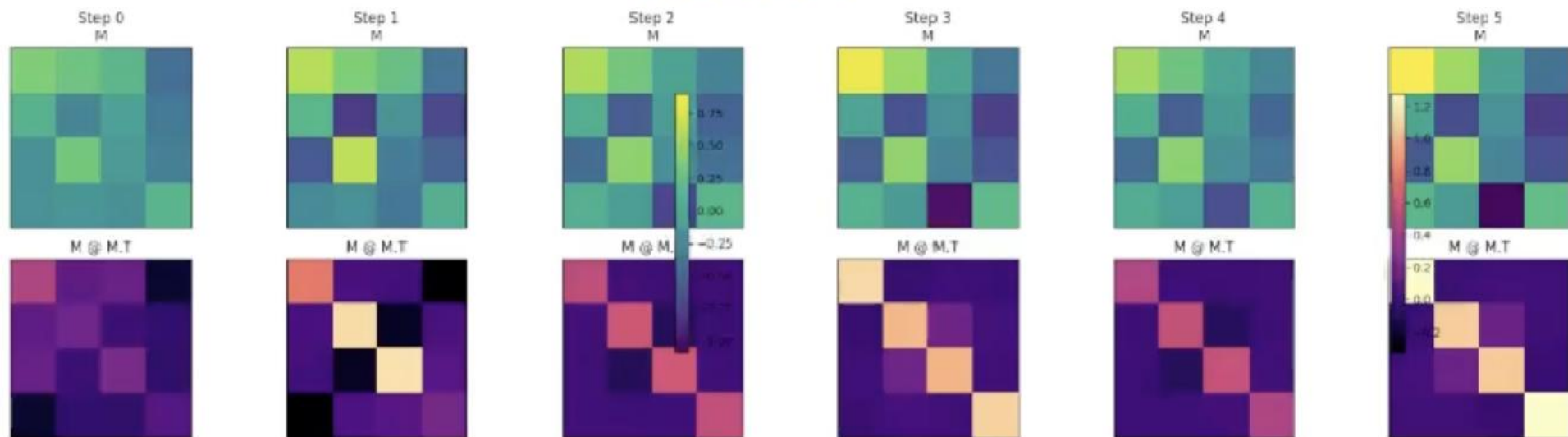
# 19 | 训练优化器的代际更迭

- Before (过去 5 年): Adam / AdamW 统治了整个 NLP 与大模型界。
- AdamW 的痛点:
  - - 内存开销: 需要保存一阶和二阶动量, 状态显存极大。
  - - 万亿参数收敛停滞: 处理 32T 数据时, 一阶近似优化器在后期出现收敛速度急剧变慢的瓶颈。
- 诉求: 需要引入二阶信息的更强优化器。

# 20 | After: Muon 优化器

- Muon 简介：一种基于正交化机制的新型优化器，专为深度神经网络的矩阵权重更新设计。
- 核心突破：相比 AdamW，Muon 引入了更接近二阶优化的曲率信息，极大提升了更新效率。
- 规模化应用：V4 首次在万亿级 (1.6T) 生产环境中全面替换 AdamW (除 Embedding/Norm 外)。
- 原作者：Keller Jordan <https://github.com/KellerJordan/Muon>

## Muon



# 21 | Muon 核心：正交正则化

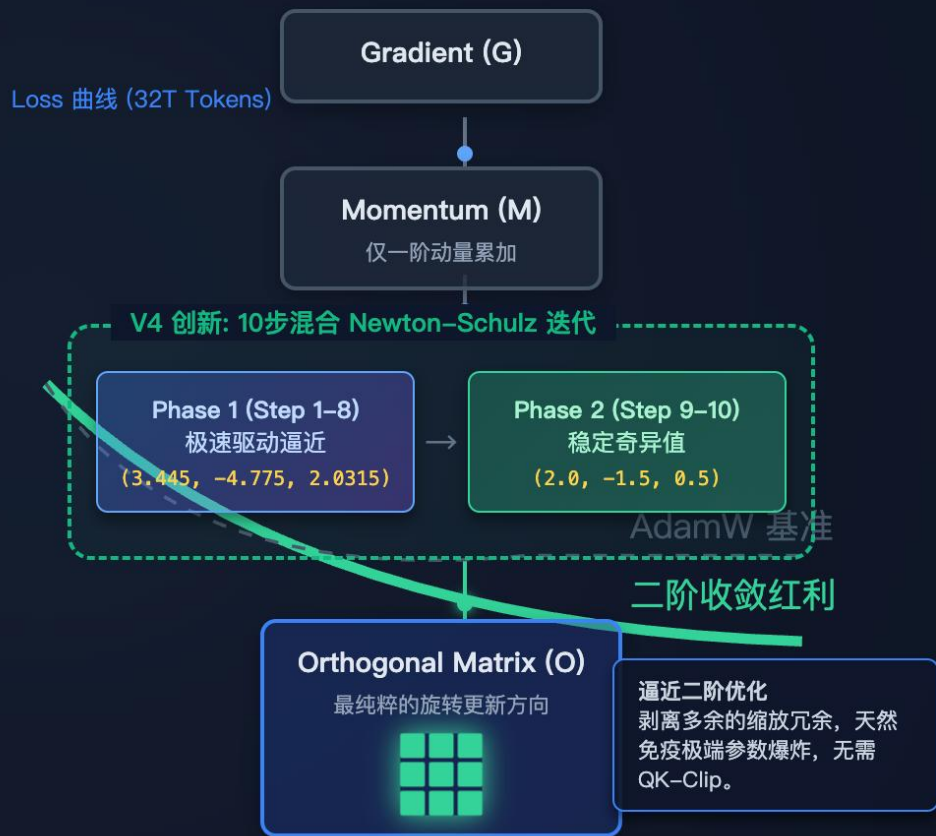
## DeepSeek-V4: Muon 优化器与混合 Newton-Schulz 迭代

正交化替代一阶动量：消除冗余缩放，获取纯粹更新方向，打破 32T 数据收敛瓶颈

### Before: AdamW 优化器



### After: DeepSeek-V4 Muon (正交正则化)

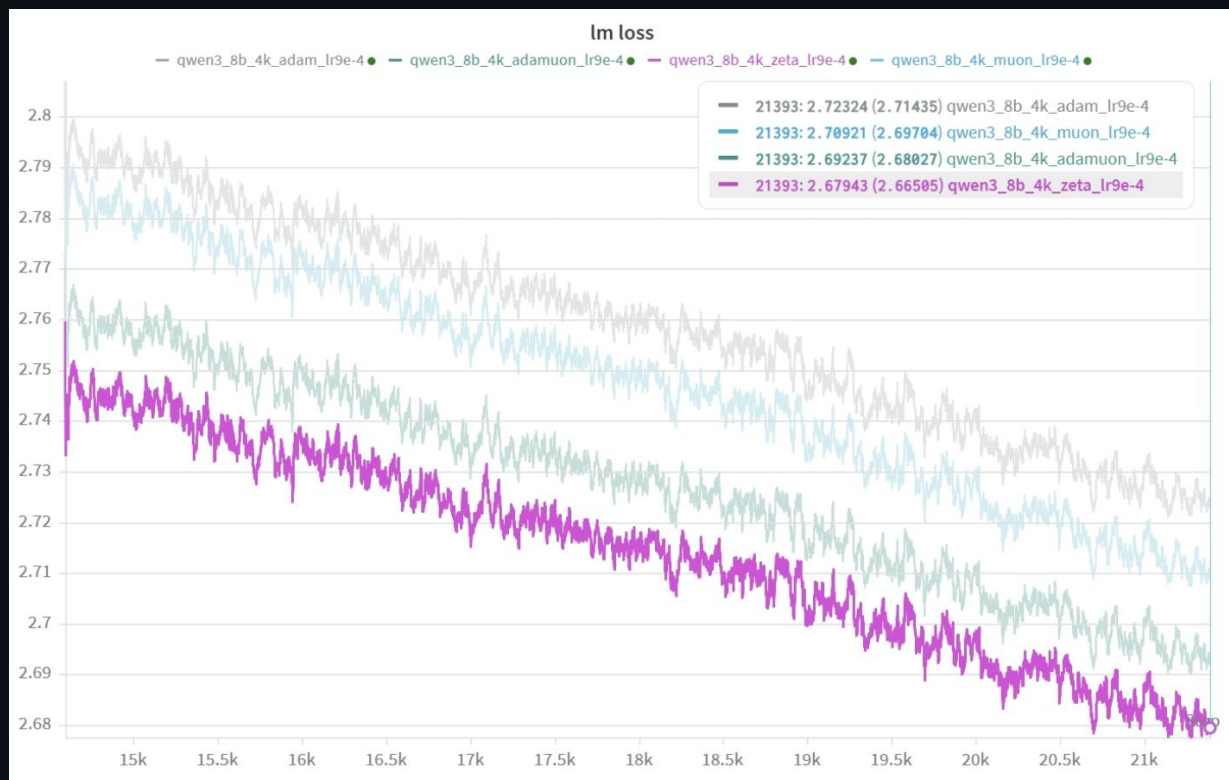


## 22 | 混合Newton-Schulz 迭代优化

- 动量处理：累加梯度生成动量矩阵  $M$ 。
- 正交化投影：不直接使用  $M$  更新权重，而是寻找最接近  $M$  的正交矩阵。
- Newton-Schulz 迭代：利用迭代法近似求解奇异值分解 (SVD)，去除冗余的缩放方向。
- 结果：仅保留最纯粹的旋转/更新方向，极大优化了高维参数空间的下降路径。
- 算法优化：V4 使用 10 步混合 Newton-Schulz 迭代。
- 第一阶段 (前 8 步)：使用驱动快速收敛的系数 (3.445, -4.775, 2.0315)。
- 第二阶段 (后 2 步)：使用稳定奇异值的系数 (2, -1.5, 0.5)。
- 优势：兼顾计算速度与极端高维矩阵下的数值稳定性。

# 23 | Muon 带来的收敛速度飞跃

- Before vs After 收敛对比:
- 在同等计算 FLOPs 下, Muon 驱动模型 Loss 下降速率显著陡峭于 AdamW。
- 物理意义: 达到相同的模型性能 (如 MMLU), Muon 需要的计算时间显著减少。
- 32T 极限压榨: 在 32T 规模下, Muon 帮助 V4 触及了前所未有的更低 Loss 下限。
- 参考我们AIGCode
- 做的优化器实验
- AdaMuon > Muon
- 但Muon收敛区问题
- So: 大概率DS
- 8步+2步解决这个问题





# 24 | 训练稳定性与消Loss Spike

- **Loss Spike (损失突刺):** 万亿模型训练的梦魇，浪费几天几夜的算力回滚。
- **Muon 的稳健性:** 由于其正交化约束，权重矩阵的奇异值被严格控制，天然免疫极端参数爆炸。
- **实战表现:** 在 DeepSeek-V4 的训练周期中，呈现出极其平滑的 Loss 曲线，几乎消灭了随机 Spike。
- 由于 V4 的 Hybrid Attention 架构直接在 Query 和 KV Entry 上应用 RMSNorm，有效阻止了 Attention Logit 爆炸。
- 因此，V4 的 Muon 优化器无需引入 QK-Clip 技术。
- 在降低计算复杂度的同时，保持了原生优化器的纯粹性。



# 25 | 优化器内存效率

- AdamW 的显存占用: 1.6T 模型仅优化器状态就需消耗 T 级别的 HBM 显存。
  - Muon 的显存挑战: 涉及矩阵正交化运算, 临时内存需求大。
  - ZeRO 并行优化: 混合 ZeRO 桶分配策略。Dense 参数限制并行度并运用背包算法; MoE 参数跨 Rank 展平对齐。
  - 结果: 峰值显存占用得到严格约束, 省出显存用于 1M Context。
- 
- 批处理不变性 (Batch Invariance): 单 Token 输出与 Batch 位置无关。放弃传统的 split-k, 自研 DeepGEMM 以保障一致性。
  - 确定性 (Determinism): 反向传播中的原子加 (atomicAdd) 导致非确定性。V4 通过每 SM 分配独立 Buffer 并在最后确定性规约求和来解决。
  - 价值: 极大方便了 Loss Spike 时的故障排查与软硬件 Debug。

# 26 | 32 万亿 (32T) 极致数据规模

- 数据量变引起质变：
  - - Llama3 (15T Tokens) -> DeepSeek-V4 (32T Tokens)。
- 质量把控：绝非爬虫堆砌，而是经过严苛洗脱的“高质量、多样化”语料。
- 核心提升：在 32T 的洗礼下，V4-Flash (284B) 获得了比肩前代超大模型的常识储备能力。
- 通用网页库：清洗去重的高质量跨语言语料，过滤掉自动生成的模板内容 (防模型崩溃)。
- 代码 (Code)：包含大量高质代码，中后期加入 Agentic 数据，强化逻辑属性。
- 数学 (Math)：提取公式、论文、深度推理步骤集，注入强推演能力。
- 长文档 (Long-Context)：大量科学论文、财报、技术报告，专为 1M 上下文打底。

# 27 | 长上下文课程学习

**挑战与 Before:** 直接用 1M 训练会导致 OOM，成本极高且难收敛；传统“一刀切”策略止步于短中等长度。

**After (V4 解法):** 采用渐进扩展策略，在特定阶段平滑过渡注意力机制，最终成功攀登 1M 巅峰。

↑ Context Length (上下文长度)

## 阶段 1: 4K 长度

前 1T Tokens

基础常识学习阶段。

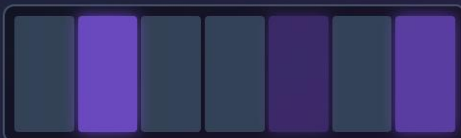
使用传统的稠密注意力 (Dense Attention)，建立模型基础语感与世界知识。



## 阶段 2: 16K → 64K

逐步扩展

开始拉长窗口，同时平滑引入 CSA/HCA 稀疏与压缩机制，让模型适应选择性遗忘与长程抓取。



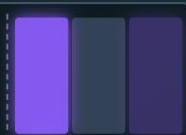
## 阶段 3: 1M Token

完全放开

架构与算力解锁。

混合注意力火力全开，利用重度压缩处理史诗级长文，成功跨越内存墙限制。

HCA 宏观



### ✗ Before (一刀切固定长度):

如果强行在初期使用长文本：KV Cache 爆炸，显存溢出 (OOM)，算力成本呈平方级

# 28 | 预防训练不稳定的“黑科技”

- Anticipatory Routing (前瞻路由):
  - - 解耦 Backbone 网络更新与 MoE 路由网络更新。
  - - 使用  $t$  步参数算特征，使用  $t-\Delta t$  步参数算路由索引，打破由于路由导致的恶性循环。
  - - 仅带来约 20% 额外开销，且只在 Loss Spike 预警时动态启用。
- SwiGLU Clamping: 将 SwiGLU 的 Linear 部分截断到  $[-10, 10]$ ，消除 Outliers。
- TileLang: 敏捷高效的 Kernel 库:
  - 由于架构极度复杂，手写数百个 CUDA 算子极不现实。
  - V4 团队采用 TileLang (DSL) 平衡开发效率与运行性能。
  - Host Codegen 减少 CPU 调度开销至  $<1$  微秒。
  - Z3 SMT Solver 引入形式化分析，确保张量切分与计算的精确无误。
  - 完全支持 FP4 乃至更低精度的量化加速 (QAT)。

# 29 | FP4 量化感知训练 (QAT)

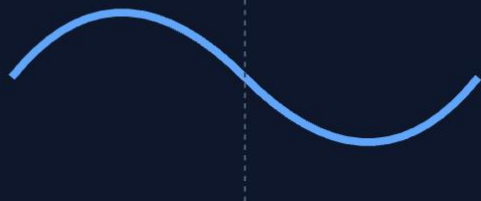
## FP4 极低比特量化：极致压缩与精度博弈

从连续的高维表示坍缩至仅 16 个离散状态，在严重精度有损下换取端侧落地可能

### BF16 / FP16 原始权重

高精度 · 连续分布 · 大内存占用

数据分布: 65536 种状态



### FP4 量化器

E2M1 / E3M0 Mapping

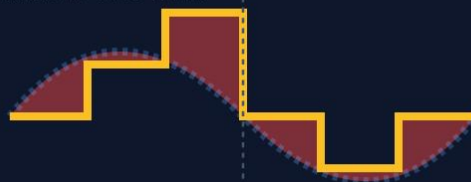


动态缩放 (Scaling)  
强制截断 (Clipping)

### FP4 量化权重

极低精度 · 阶梯分布 · 显存降 75%

数据分布: 仅 16 种状态



⚠ MSE 飙升

### 严重的 Loss 及精度有损

FP4 (如 E2M1) 全局仅能表示 16 个具体的数值。在图表中表现为强烈的阶梯状离散跳变。

这导致原本平滑的权重分布产生巨大的 MSE (均方误差)。在通用训练或

### 端侧推理 (Edge AI) 核心红利

尽管精度损失惨重，但 FP4 将模型体积压缩了 75% 以上，并极大缓解了内存带宽 (Memory-Bound) 瓶颈。

核心价值：让原本无法在手机、IoT 端侧运行的超大模型（或千亿级

# 30 | 综合后训练管线

- V4 哲学：预训练注入知识，后训练解锁 (Unlock) 和增强 (Enhance) 能力。
- 两阶段管线：
  1. 领域专家培育 (Specialist Training): 通过 SFT 和 GRPO 分别训练数学、代码、指令遵循等专家模型。
  2. 同策略蒸馏 (On-Policy Distillation, OPD): 将多个老师模型的能力合并到一个学生模型 (统一底座)。
- GRPO 与生成式奖励模型 (GRM)
- GRPO: 免除显存杀手 Value Model, 采用组内相对奖励优化。
- 生成式奖励模型 (GRM):
  - 针对难以用规则验证的复杂推理任务。
  - 放弃标量 Reward Model, 直接让 Actor 网络兼任裁判 (Judging)。
  - 内部推理能力自然融入评估过程, 抗欺骗能力极强。



# 31 | 多阶段思维 (Interleaved Thinking)



# 32 | 多阶段思维 (Interleaved)

- 在 Agent 长程任务中保存完整思考链路。
- **Before (V3.2):** 新用户输入会清空之前的 **<think>** 内容。
- **After (V4):** 得益于 1M 上下文, 工具调用 (Tool-Calling) 场景下完全保留所有轮次的思维痕迹。
- 效果: 实现连贯的长期规划与自我修正。
- 多教师同策略蒸馏 (OPD):
- 融合专家能力: 十余个千亿/万亿级专家模型共同指导一个 V4 模型。
- 全词表 Logit 蒸馏: 不采用低效的单 Token 估算, 保留全词表分布计算逆 KL 散度, 确保梯度稳定。
- 工程挑战: 海量教师模型的显存占用极大。
- 解法: 后台异步拉取, 按教师重排 Batch, 并仅缓存倒数第二层 Hidden States 以实时重建 Logit, 极大节约显存。



# 33 | V4-Pro-Max 极致模式

- Pro-Max 登场: V4-Pro 架构下的最强形态 (Maximum Reasoning Effort)。
- 核心设计: 注入极高要求的 System Prompt, 放开思考 Token 限制, 强制分解问题和穷尽所有假设。
- 突破极限: 彻底释放 1.6T 模型的推演算力。
- 超越开源: 在数学、代码奥赛任务中打败所有开源模型, 比肩甚至部分超越闭源 o1/Gemini SOTA。
- 世界知识: SimpleQA-Verified 准确率大幅领先开源界 20%。
- 复杂推理: Math、AIME、CodeForces 评分达到人类 Top 水平, 逼近闭源前沿。
- 模型对比:
  - - V4-Pro-Max 全面统治开源 SOTA。
  - - V4-Flash 以 13B 激活参数超越了体量庞大的 V3.2。

# 34 | 1M 大海捞针及实测

- MRCCR / CorpusQA 评测：在 1,000,000 Token 长文本库中寻找孤立线索。
- 无损压缩证明：尽管 HCA 将远端文本极度压缩 ( $m'=128$ )，V4 在 1M 长度下依然保持极高的 MMR 与 ACC 召回率。
- 超越竞品：长文本实测性能显著超越 Gemini-3.1-Pro。
- 意义：长文本不仅能“放进去”，而且真能“找出来”。
- 中国职场全景测试 (White-Collar Task):
  - - 覆盖 13 个行业的分析、生成、编辑任务。
  - - V4-Pro-Max 的非败率高达 63% (对比 Opus-4.6-Max)，尤其在深度信息分析上优势明显。
- Code Agent:
  - - 内部研发环境实测解决真实 Bug，52% 工程师认为其可作为首选编码助手，部分逼近闭源 SOTA。

# 35 | 结语：百万上下文普惠时代

- **【三大极致】**
  - 1. 极致压缩：CSA+HCA 打破了长文本的内存墙。
  - 2. 极致稳定：mHC 超连接解决了 1.6T 万亿网络的流形坍缩。
  - 3. 极致收敛：Muon 优化器在 32T 数据上实现了降维打击。
- 在已知研究所得范围内，做了工程化的强实现（混合注意力，后训练放弃reward，符合我之前说的后训练太多因子其实是干扰）。
- 预训练方面的基础研究创新略少或者说严谨偏保守：如RoPE，Muon(SOTA是AdaMuon)优化微创新，后训练微创新。
- **【最终愿景】**
  - DeepSeek-V4 是底层架构的胜利，它让百万级上下文大模型正式走向平民化与生产力时代！